

Dynamic Difficulty Adjustment in Video Games: A Design Pattern Based Framework for Enhanced Gaming Experience

Muhammad Ehsan Rana

*Asia Pacific University of
Technology and Innovation Technology*
Bukit Jalil, Kuala Lumpur, Malaysia

Mohanad Qusay Atta

*Asia Pacific University of
Technology and Innovation Technology*
Bukit Jalil, Kuala Lumpur, Malaysia

Tan Chin Ike

*Asia Pacific University of
Technology and Innovation Technology*
Bukit Jalil, Kuala Lumpur, Malaysia

Abstract—The emergence of Dynamic Difficulty Adjustment (DDA) has revolutionized player experience in video games by addressing challenges related to boredom and frustration. DDA balances game difficulty, taking into account players' skills and progress. However, implementing DDA in video games is a complex endeavour, entailing challenges and costs similar to those encountered in the software industry. In this paper, the authors presented an object-oriented design framework that incorporates selected design patterns from the Gang of Four, empowering the application of DDA in object-oriented games. This research extensively explores the perspectives of players/gamers and game software developers to effectively develop the framework. By utilizing object-oriented design patterns, the resulting framework provides a flexible and adaptable solution for integrating crucial DDA elements. With minor modifications, the design can be tailored to accommodate diverse requirements. This research contributes to the advancement of game development practices by providing a robust and practical framework for applying DDA, ultimately enhancing the gaming experience and player satisfaction.

Index Terms—Video Games, Dynamic Difficulty Adjustment (DDA), Gaming Experience, Object-Oriented Design Framework, Design Patterns

I. INTRODUCTION

The video game industry has become a highly lucrative sector within the entertainment domain, captivating millions of players worldwide [1]. With various platforms available, such as smartphones, computers, and gaming consoles, players have numerous avenues for engaging and entertaining themselves. The advent of IoT, mobile computing, augmented reality, and other technological advancements has further enabled players to compete with opponents from different regions, showcasing their remarkable abilities. In this highly competitive and ever-evolving market, ensuring positive player experience and satisfaction is crucial for the growth and survival of video game companies [2].

One traditional and widely employed approach to enhance user satisfaction is by adjusting the difficulty level of games. This component plays a vital role in determining the fun factor [3]. Achieving an optimal difficulty level requires meeting several parameters, including players' learnability,

interaction with environmental objects within the game, and the challenges presented at different levels. The progression of player skills and their ability to advance in the game are significant factors influencing the usability of computer systems [4]. The gameplay in video games encompasses a set of patterns and rules that determine the difficulty level and overall player experience. Traditionally, game developers have relied on preset difficulty settings either at the start or during gameplay. This approach continues to be utilized across various genres such as First-Person Shooters (FPS), Third Person Shooters (TPS), Adventure, Strategy, Sports, and Indie titles [citation]. However, this fixed approach may not always provide the optimal level of satisfaction and entertainment that players seek. An alternative method to control the gaming experience is through Dynamic Difficulty Adjustment (DDA), which monitors players' difficulty levels in real-time and dynamically adjusts them using various algorithms. Unlike fixed difficulty settings, DDA can prevent players from becoming frustrated by offering a more balanced and engaging gameplay experience [5]. Games such as FIFA under the Electronic Arts (EA) Sports franchise are a perfect example of a video game that uses DDA to monitor a player's skill and ability and provides feedback to the player to either increase or decrease the difficulty levels to have a better overall game experience. In this research, the authors proposed a comprehensive yet generic framework that incorporates selective design patterns for dynamic difficulty adjustment (DDA). These design patterns are chosen based on their suitability and effectiveness in enhancing the design elements of reusability, reliability, and efficiency. By integrating these patterns into game development, a flexible and adaptable framework is developed that can enhance the overall gameplay experience for players.

II. LITERATURE REVIEW

The most commonly used options for specifying difficulty levels in video games are Easy, Normal, and Hard. Each option represents a predefined set of settings that determine the overall difficulty of the game as players begin their gaming experience. The level of difficulty directly impacts how easy or challenging the gameplay will be for the player,

making it a crucial element in creating an enjoyable gaming experience [6]. Fun is the primary key element in successful entertaining games, as emphasized by Andrew and Morris, excluding educational or special-purpose games like military simulations [7]. While fixed difficulty levels like Easy, Normal, and Hard are commonly used, simply adding options like "Very Easy" or "Very Hard" does not necessarily provide the desired satisfaction for players. This is because gamers have diverse skill levels and varying abilities when playing the same game.

A. The Need for Dynamic Difficulty Adjustment (DDA)

To achieve a high level of player satisfaction, thus assuring the viability and success of a video game title is to always ensure that there is positive player engagement for sustained play [8]. According to Starks [29], the phenomenon described as flow is one of the most effective methods of ensuring players attain a positive and engaging experience by combining motivation and the level of achievement. Flow is a condition in which both a player's skill and the challenge are evenly matched, resulting in the player being totally immersed in achieving the goals and objectives in the game or overcoming the challenges or levels in the game and eventually losing track of time [30]. This is true player immersion in video game parlance. To ensure the delicate balance of flow is met, a dynamic approach that offers varying difficulty levels has been developed. This dynamic difficulty adjustment (DDA) adapts the game's challenges in real-time based on the player's skill level. Considering each player's capabilities and how they can advance their skills are crucial factors in the usability of computer systems, as argued by [4]. By implementing DDA for game AI, the game experience can be dynamically and in real-time tailored to the player's skill level, enriching the overall gaming experience and providing as much fun as possible. With DDA, the game adapts to the player's skill level, ensuring a challenging yet enjoyable experience. It allows players to progress and improve their skills while maintaining engagement and preventing frustration. By dynamically adjusting the difficulty, game developers can cater to the individual needs and preferences of players, enhancing the overall appeal and longevity of the game [9].

B. Implementing DDA: Adapting Gameplay in Real Time

Despite advancements in the gaming industry, many game developers still rely on fixed difficulty options or settable difficulty levels in their games. Even popular developers and publishers, such as Activision's Call of Duty Black Ops, continue to use this approach. However, as discussed by [10], this approach can often lead to overwhelming or boring gameplay experiences. Even if players can manually adjust the difficulty during gameplay or if the game implements dynamic difficulty adjustment (DDA), simply changing the difficulty level without considering the player's skills in real-time can negatively impact the overall fun and enjoyment of the game. It's important to remember that the primary purpose of these types of games, such as action/adventure or racing games, is to

provide fun and entertainment, which is the main reason why people buy video games in the first place [11]. While some game developers have attempted to address the challenge issue by adjusting certain elements such as increasing or decreasing the player's character attributes like health or ammo, these solutions only address non-dynamic aspects of the game and fail to fully solve the problem [12],[13]. To truly enhance the game experience, there is a need for an approach that supports the use of DDA while also incorporating classical difficulty adjustment. This approach should take into account additional factors and attributes that have not yet been highlighted by these two existing approaches. By considering a broader range of factors and attributes, game developers can create a more immersive and engaging experience for players. For example, instead of solely focusing on changing numerical values like health or ammo, other important elements such as enemy behaviour, level design, or environmental factors can be dynamically adjusted to provide a more tailored and enjoyable gameplay experience. By embracing a holistic approach that combines DDA and classical difficulty adjustment, game developers can unlock new possibilities to enrich the game experience and cater to the diverse skill levels and preferences of players [14].

C. Potential Benefits and Challenges Associated with DDA

In the realm of Dynamic Difficulty Adjustment (DDA), there are both supporters and individuals with concerns about this approach. These findings shed light on the potential benefits and challenges associated with implementing DDA in video games.

1) *Potential Benefits:* adjustments. For example, Arulraj (2010) proposed the use of dynamic scripting and machine learning to generate adaptive agents, which he claimed would effectively improve DDA efficiency in games. [10] found that adjusting the simulation time of Monte Carlo Tree Search (MCTS), a technique for generating opponent AI, along with DDA, led to an improved game experience and increased player satisfaction. [15] also concluded that utilizing DDA primarily based on MCTS could provide a positive game experience. These supporters focused on applying DDA to various elements and attributes within games, such as modifying the rate of fire of NPCs in shooter games, adjusting player and NPCs' health, and altering the frequency of opponent respawns.

2) *Challenges:* Some individuals express concerns about applying DDA in video games, and their claims typically fall into two domains.

Critics argue that implementing DDA can increase the complexity and cost of game development. Developing a game is already a challenging and costly endeavour, and some argue that it may be easier and more cost-effective for game developers to utilize settable difficulty options instead of integrating DDA [16].

Another set of concerns revolves around resource consumption, particularly in terms of hardware system resources like CPU and memory. Applying DDA often involves real-time

observation and decision-making based on significant events that trigger changes within the DDA system. Critics worry that such real-time processing may place additional strain on system resources, potentially affecting game performance [15].

3) *Summary:* In summary, the literature findings reveal that while there are supporters of DDA who emphasize its potential to enhance the game experience through adaptive adjustments, there are also concerns about the complexity and cost of development, as well as potential resource consumption and technical issues. Addressing these concerns and finding ways to mitigate them would contribute to a more comprehensive understanding and successful implementation of DDA in video games.

D. Research Gap

The current state of the literature on Dynamic Difficulty Adjustment (DDA) reveals several gaps and areas that need improvement. After analysing academic sources that study DDA and its applications, it becomes evident that there is a lack of a practical supporting environment to facilitate DDA development in video games. Furthermore, there is a dearth of literature within the academic community that addresses the specific attributes or elements that can be dynamically altered to increase or decrease game difficulty through DDA. These gaps can be attributed to the complexity of game development and the absence of proposed technical solutions that can simplify and support the implementation of DDA in object-oriented games. The lacking areas can be summarized into two categories:

1) *Lack of DDA Frameworks Focusing on Technical Aspects:* While DDA significantly impacts gameplay, it is primarily the game designers who engineer and design the gameplay. As a result, most studies and discussions about DDA tend to focus on the design aspects. However, programmers and software engineers/developers are responsible for translating the designers' vision into a functional product. Applying DDA introduces its own set of risks and complexities, which may increase the development process's complexity and costs. Therefore, the available literature on DDA needs to be enriched with technical aspects, such as frameworks that can facilitate the smooth implementation of DDA [17].

2) *Lack in the Game Design Perspective:* There are notable gaps in the research supporting DDA from a game design perspective. Currently, modifying non-player character (NPC) artificial intelligence (AI) using DDA is done on generic attributes of the game's opponents/NPCs, rather than on an individual basis. Consequently, modifications are applied uniformly, resulting in a group of similar opponents behaving identically. For example, in a first-person shooter (FPS) game utilizing DDA, if the opponents' AI is modified to increase the rate of fire, all opponents of the same type will exhibit the same behaviour [18], [19].

3) *Limitations of DDA in Emulating Real-Life Attributes:* While dynamic difficulty changes provide some improvements to the game experience and its enjoyment, it does not necessarily deliver optimal enhancements. Most opponents in games

are designed to emulate real-life attributes and characteristics defined during the game design stage. For instance, in a warfare game, each opponent aims to simulate the behaviour of a real soldier. It is important to acknowledge that individuals within a group, even if they belong to the same category, possess unique capabilities, skills, and attributes. These attributes include rank (higher-ranked soldiers may be more skilled professionals), experience gained from service duration, physical fitness, intelligence, bravery, and more. To achieve a more realistic and enjoyable game experience, it would be beneficial to consider these attributes when simulating opponents. DDA can dynamically modify the game difficulty, but it also needs to incorporate these attributes as initial values alongside other factors that DDA considers when deciding whether to increase or decrease difficulty. Therefore, there is a need for a solution, represented by an algorithm or technique, to provide the initial state of DDA with these additional attribute values, enhancing the manipulation of game difficulty. This capability could be implemented as a plug-in, requiring minimal modifications to the existing solutions mentioned above. Furthermore, it could also support the classic approach of fixed difficulty options, depending on how opponent AI is designed and the desired difficulty adjustments and key gameplay attributes [9], [20].

E. Significance of Research

Dynamic gameplay that enhances the user experience is a crucial factor in game development. Various approaches have been proposed to improve gameplay and make games more enjoyable, and Dynamic Difficulty Adjustment (DDA) is one such approach. DDA enables real-time decision-making and control based on player skills [21]. However, implementing DDA can be challenging due to the limitations and difficulties faced by the game industry. Although researchers have concluded that DDA is a viable approach, the lack of industry adoption and practical values remains an obstacle. Many research studies in this area have been conducted on older two-dimensional games as proof of concept, while the game industry continues to predominantly use classical approaches for settable difficulties in modern games, with a few exceptions [10]. This research aims to bridge the gap in the field by proposing a practical framework that addresses both sides of the DDA debate. By providing an example of Dynamic Difficulty Adjustment that manipulates gameplay elements such as Pathfinding Algorithms in FPS games using DDA, the research takes into account industry needs and values, including the philosophical perspective of player-game interaction and the endorsement of DDA concepts by researchers. The contribution of this dissertation will enrich the available context and provide feasible solutions, particularly for smaller developers and indies who may have limited technical support and rely on major publishers' in-house development tools considered their trade secrets. By developing a framework and addressing the challenges and limitations surrounding DDA, this research aims to support the implementation of DDA in games and facilitate the creation of dynamic and engaging gameplay experiences. It seeks to demonstrate the

value and practicality of DDA, ensuring that it aligns with industry requirements and promotes the overall enjoyment and satisfaction of players.

III. OBJECT-ORIENTED DESIGN PATTERNS FOR DYNAMIC DIFFICULTY ADJUSTMENT IN VIDEO GAMES

The object-oriented paradigm plays a fundamental role in the development of large and complex systems, offering a set of principles that facilitate the creation of practical solutions within feasible time and budget constraints [22]. However, achieving essential principles such as reusability, encapsulation, and inheritance can present challenges, as noted by [23]. To address these challenges and optimize system design, researchers have turned to design patterns, which provide specific approaches and effects applicable in diverse scenarios. Design patterns are standardized solutions to common software design problems that have been extensively tested and proven effective over time. They provide a structured approach to solving recurring design challenges and promote code reusability, maintainability, and flexibility in software development. Design patterns are essential in the object-oriented paradigm, where they help in creating efficient, scalable, and robust systems. There are various types of design patterns, including creational patterns (e.g., Singleton, Factory), structural patterns (e.g., Decorator, Adapter), and behavioural patterns (e.g., Observer, Strategy). Each design pattern addresses a specific design issue and offers a set of guidelines and best practices to achieve the desired functionality [24], [25]. The integration of design patterns and DDA in video games provides numerous advantages. Firstly, design patterns promote maintainability by facilitating modular and structured code, making it simpler to extend and manage the game's features, including DDA functionality. Secondly, design patterns encourage code reusability, enabling game developers to apply proven solutions to similar DDA challenges in different games, thereby enhancing development efficiency. Lastly, utilizing design patterns leads to time and cost efficiency, as developers can leverage established solutions, avoiding redundant work and benefiting from proven practices [26], [27]. Design patterns and Dynamic Difficulty Adjustment are indispensable components in modern game development. By leveraging design patterns to implement DDA, game developers can create dynamic and immersive gaming experiences that cater to a wide range of player skill levels. The synergy between these concepts empowers game designers to build flexible and adaptive games that consistently engage and delight players, further enhancing the ever-evolving landscape of the gaming industry [28]. This study focuses on the Observer, Strategy, and Decorator design patterns, examining their potential for enhancing Dynamic Difficulty Adjustment (DDA) in Video Games. The objective is to evaluate the suitability of these design patterns for implementing DDA in video games, where the game's difficulty dynamically adjusts according to the player's skills and progress. By investigating the Observer pattern, which facilitates the monitoring and updating of player and Non-Playable Character (NPC) attributes, the Strategy

pattern, which enables flexible and interchangeable algorithms for AI behaviours, and the Decorator pattern, which enhances object functionalities dynamically, this research seeks to enhance the gaming experience through intelligent and adaptive gameplay.

IV. PROPOSED FRAMEWORK

This research presents a comprehensive framework aimed at supporting Dynamic Difficulty Adjustment (DDA) in Video Games. The proposed framework takes advantage of industry-proven object-oriented design patterns as a solid foundation for development, addressing the technical challenges associated with implementing and applying DDA. By utilizing these design patterns, the framework can be universally applied to various software systems, including computer games, as they effectively define software components that synergistically create highly interactive media systems. This paper explores the potential of object-oriented design patterns to enhance the gaming experience by dynamically adjusting difficulty levels, ultimately offering improved and engaging gameplay for players. This section illustrates the framework's main components (selected patterns) that are expected to provide a practical foundation for applying Dynamic Difficulty Adjustment in video games.

A. Observer Design Patterns

At the heart of the proposed framework lies the Observer pattern, also known as the Publish-Subscribe and Dependent pattern. This pattern establishes a dependency relationship among a group of classes, where certain classes continuously observe the status of a target class or its representative object. Based on the state of this targeted component, the observing class then updates other related (subscribed) classes on the status changes. As described by [23], the Observer pattern defines the nature of the relationship between objects, specifically a "One to Many" relationship. In this way, objects can automatically update their states when another object undergoes a state change. This powerful pattern allows for dynamic communication and coordination among various components in the game software, facilitating real-time updates and reactions to changes in the game environment or player actions. The Observer pattern plays a crucial role in the implementation of Dynamic Difficulty Adjustment (DDA) in video games, as it enables the monitoring of player states and triggers adjustments in response to their performance. By employing the Observer pattern, game developers can efficiently manage and synchronize the interactions between different game elements, enhancing the overall gaming experience and ensuring seamless gameplay adaptability. The Observer design pattern is transformed to fit the current and future technical requirements of game development so that its implied concept is adopted to serve the special nature of game development within game engines for both commercial (e.g. Unity3D) and proprietary (Digital Illusion CE's Frostbite 2, owned by Electronics Arts EA) engines. Each engine has some technical variables and coding/scripting standards that contain

predefined concepts (especially for object-oriented approach) that researchers have tweaked to fit the game development scenario, for instance, the way of initializing variables and its constructing and deconstructing, memory access constraints and specifiers, etc. Such differences need to be considered when the framework is designed.

1) *Intent*: The observer pattern plays a crucial role in action games, particularly First-Person Shooter games, by closely monitoring the player's object, which represents the playable and controllable character. The dynamic and engaging gameplay heavily relies on interactions, conflicts, and engagements between the player and Non-Playable Characters (NPCs). These NPCs serve as the primary interactive components of the game, and their behaviours can be classified into multiple categories. However, the focus lies on technical and programmable behaviours, often created using complex artificial intelligence (AI) algorithms or neural networks. Notably, AI-driven aspects such as enemy awareness, pathfinding, and level navigation significantly influence the gameplay, adding depth and challenge to the gaming experience. On the artistic side, various behaviours are responsible for enhancing the visual and emotional aspects of the game. These behaviours pertain to character animations, including walking, running, jumping, shooting, and more. Additionally, facial expressions play a crucial role in conveying emotions effectively, such as synchronizing a character's lips with vocals or displaying expressions like smiles and anger. To achieve these visually appealing behaviours, game developers utilize artistic tools like Autodesk Maya, Blender, and other modelling packages. Moreover, in-engine tools, such as Unity3D's animation tool, are commonly employed to fine-tune imported animations or create simpler animations to enrich the overall gaming experience. By integrating these artistic elements, games attain a heightened level of realism and engagement, complementing the technical aspects driven by AI and the observer pattern.

2) *Problem Context*: It is essential to recognize the significance of both distinctive behaviour generation approaches in game development. While artistic behaviours contribute to the visual appeal and emotional immersion of the game, the technical and programmable behaviours of Non-Playable Characters (NPCs) take centre stage in this research. Specifically, the focus is on leveraging these technical behaviours, driven by AI algorithms or neural networks, to achieve Dynamic Difficulty Adjustment (DDA) based on player attributes. By fine-tuning the NPCs' responses and challenges in real-time according to the player's skills and progress, the DDA enriches the gaming experience, ensuring that the game's difficulty remains engaging and enjoyable for players of varying expertise levels. The harmonious integration of both artistic and technical aspects helps realize the creative vision of the designers and creative directors, culminating in a compelling and immersive gameplay experience. The Observer Pattern serves as a vital tool to manage various dynamic and adjustable characteristics of both the Non-Playable Characters (NPCs) and the player. These attributes encompass elements such as health, ammo, and any other factors influencing gameplay difficulty, making

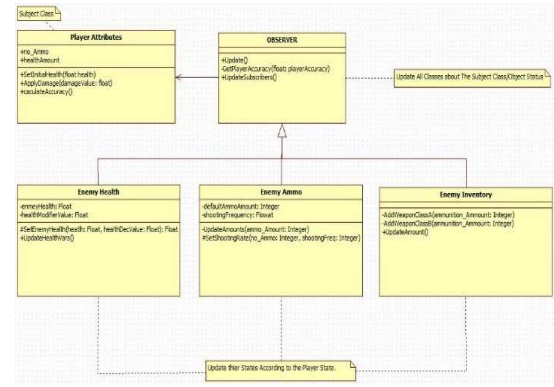


Fig. 1. Observer Pattern Class Diagram for DDA

them instrumental in the game difficulty balancing process. Additionally, the Observer Pattern is adept at handling time-related challenges, such as assessing the 'required' versus 'available' time for players to complete specific gameplay tasks within designated timeframes. By closely monitoring and updating these attributes, the Observer Pattern plays a crucial role in ensuring a dynamically challenging and engaging gaming experience for players, adapting to their skills and pace as they progress through the game.

3) *Solution*: The class diagram (Fig. 1) illustrates a representation of the observer pattern for Dynamic Difficulty Adjustment. In this design, the player's attributes represent the observed object, while the Observer class will notify the Enemy Health, Enemy Ammo, and Enemy Inventory classes about the player's states. Consequently, the health, ammo, and type of weapons the enemy uses will be adjusted based on the player's skills. This illustrates how the Observer Pattern can effectively facilitate Dynamic Difficulty Adjustment (DDA) by continuously monitoring and updating the states of the player and the Non-Playable Characters (NPCs), allowing for seamless and engaging gameplay experiences.

4) *Consequences*: Dynamic Difficulty Adjustment (DDA) plays a vital role in managing and applying adjustments to various attributes based on the player's statistics and characteristics, including adjustable factors like health and resources. By gathering data from the player's statistics, DDA intelligently modifies the attributes of Non-Playable Characters (NPCs), ensuring a challenging yet enjoyable gaming experience. This process aligns with the essence of the Observer Pattern, where the DDA continuously monitors the player's object status and updates the states of subscribed entities, such as NPCs, accordingly. In essence, the Observer Pattern serves as a technical design strategy and a real-world concept, enabling the seamless integration of DDA as a dynamic and adjustable attribute of the player's object, which effectively updates and adapts the attributes of opponents in response to the player's skills and gameplay progression.

B. Strategy Design Pattern

The Strategy Pattern serves as a valuable tool for encapsulating multiple algorithms and dynamically substituting

them in real-time. By reducing the dependency of system components on specific algorithms and their instances, this pattern enables these algorithms to be used by other system components and classes, promoting flexibility and adaptability in the system's design [23]. In the context of Dynamic Difficulty Adjustment (DDA) in video games, the Strategy Pattern proves to be particularly beneficial. It provides a solid object-oriented foundation that supports the implementation of DDA, which significantly influences the game design domain. By effectively coordinating technological capabilities with the creative vision of game design concepts and gameplay mechanics, the Strategy Pattern empowers game developers to enhance the overall fun and enjoyment of the game—a pivotal factor in its success. By employing the Strategy Pattern, game developers can seamlessly integrate and switch between different difficulty adjustment algorithms, tailoring the gameplay experience to the individual player's skill level and progress. This adaptability allows for a more engaging and immersive gaming experience, as the game dynamically adjusts its challenges to match the player's abilities. As a result, the Strategy Pattern contributes to creating games that captivate and entertain players of various skill levels, ultimately leading to greater player satisfaction and commercial success.

1) *Intent*: Certain video games demand advanced and expensive hardware resources, necessitating high processing power and significant memory usage. Among the most resource-consuming computational algorithms are pathfinding algorithms, such as A* and Dijkstra's algorithms. These algorithms play a critical role in guiding objects to navigate through the game's environment, and their execution can be repeated multiple times for dynamic objects with continuous movements. Consider a two-dimensional (2D) environment where objects need to accelerate from one point to a target location, or an enemy navigating through the space with both X and Y-axis movements. Similarly, in three-dimensional (3D) space, objects may climb hills or manoeuvre through bumpy terrain, requiring compositional movement along the X, Y, and Z axes. In such cases, pathfinding algorithms must continually search for the optimal path between the current position and the target position, depending on the specific algorithm employed. Due to the dynamic nature of game objects, they constantly seek their objectives and are not static entities. As a result, even simple movements for non-playable characters (NPCs) can trigger multiple executions of pathfinding algorithms. These repeated computations arise from the need to efficiently guide dynamic objects through the game environment, creating engaging and interactive gameplay experiences for players. As game developers strive to strike a balance between realism and performance, optimizing pathfinding algorithms becomes a critical aspect of video game design and development.

2) *Problem Context*: Implementing Dynamic Difficulty Adjustment (DDA) in video games involves addressing various challenges related to pathfinding accuracy, algorithms, and opponent speed. Additionally, managing the distance between the player and the respawn points presents unique difficulties.

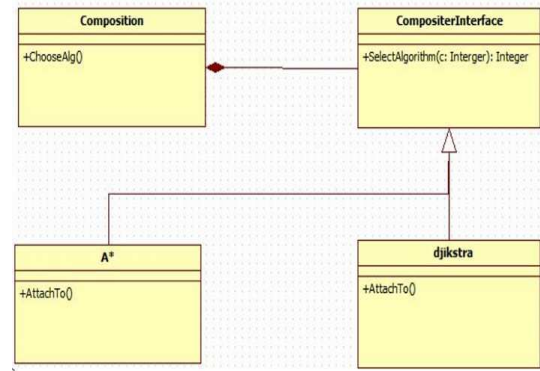


Fig. 2. Strategy Pattern Class Diagram for DDA

Pathfinding accuracy and algorithms play a crucial role in determining the opponents' behaviour and movements. Fine-tuning these aspects to achieve a balanced and challenging gameplay experience can be complex. The speed at which opponents locate the player is also a critical factor in maintaining the game's difficulty level. Adjusting these parameters dynamically in response to the player's skills and progress requires careful consideration and testing. Respawn points are predefined locations in the game world where new Non-Playable Characters (NPCs) are created as game objects to re-enter the gameplay. This mechanism ensures that the player encounters fresh challenges even after defeating opponents. However, managing the placement and timing of respawn points to maintain an engaging gameplay experience can be quite challenging. Care must be taken to position respawn points strategically, allowing NPCs to appear in locations where the player may not anticipate them. This surprise element enhances the game's dynamics and keeps players immersed in the experience. Furthermore, ensuring that newly created NPCs navigate efficiently to their intended targets, which could be the player or strategic locations for engagements, is essential for a smooth and challenging gameplay experience. The game system must handle these respawned NPCs' movements seamlessly, ensuring they add to the game's excitement without causing disruptions or inconsistencies.

3) *Solution*: The Strategy pattern proves to be a valuable asset in the game system, allowing it to dynamically change algorithms based on feedback received from the Observer Pattern within the proposed framework. Leveraging this pattern can significantly support the implementation of techniques like Dynamic Difficulty Adjustment (DDA) in video games, offering practical and effective support. Fig. 2 showcases the application of the Strategy pattern for Dynamic Difficulty Adjustment, derived from the original Design by the Gang of Four. This illustration depicts how the Strategy pattern enables the integration of various pathfinding algorithms seamlessly. It highlights the system's capability to switch from resource-intensive algorithms to more efficient ones, optimizing computer resource consumption.

4) *Consequences*: The Strategy pattern plays a crucial role in enhancing Dynamic Difficulty Adjustment in video games. By allowing the game manager to adapt algorithms based on feedback from the Observer Pattern, the game system becomes more responsive and adaptable to player interactions. The Strategy pattern's versatility empowers developers to implement multiple pathfinding algorithms efficiently, optimizing the game's performance and overall gameplay experience. As game development continues to evolve, the integration of the Strategy pattern proves to be a valuable and practical solution for achieving engaging and dynamic gameplay.

C. Decorator Design Pattern

The decorator pattern serves as a valuable alternative to subclassing when adding new functionalities dynamically to an object. In the realm of computer games, this pattern proves to be instrumental in developing in-game features that require flexibility and extensibility [23]. In the context of computer games, numerous aspects can benefit from the Decorator pattern. For instance, consider the scenario of unlocking advanced weapons after upgrading a player's points. By applying the Decorator pattern, new functionalities can be seamlessly added to the player's arsenal, enhancing their gaming experience. For example, additional components like scopes, silencers, and grappling tools can be dynamically incorporated into the player's weapons, allowing them to adapt and strategize based on their preferences and evolving gameplay. The Decorator pattern's versatility in game development ensures that new features can be integrated without the need to create multiple subclasses, simplifying the design and maintenance process. This approach fosters a more modular and flexible architecture, making it easier to manage in-game features and promoting a seamless and enjoyable gaming experience for players. As game development continues to evolve, the Decorator pattern proves to be an invaluable tool for enriching gameplay and providing players with a wide range of engaging and customizable experiences.

1) *Intent*: In the realm of difficulty balancing, the Decorator pattern emerges as a pivotal tool for dynamically adjusting the features of Non-Playable Characters (NPCs) in a video game. By seamlessly adding new features like weapons, grenades, and other elements to NPCs, the Decorator pattern effectively raises the gameplay's difficulty, ensuring an engaging and challenging experience for players. Consider a scenario where a patrol of NPCs, consisting of several soldiers, engages the player in combat. If the player successfully eliminates the entire patrol, the Decorator pattern can come into play by dynamically assigning more powerful weapons or additional responsibilities to the respawned NPCs. As a result, these NPCs now possess enhanced capabilities, such as wielding RPG7s or grenades, as they relentlessly target the player. This dynamic adjustment in the NPCs' attributes ensures that the game remains challenging even as the player progresses and adapts to the gameplay. The Decorator pattern's ability to dynamically add new features without altering the existing NPC classes simplifies the process of difficulty balancing. Game

developers can iteratively enhance the challenges faced by players by assigning more responsibilities or features to NPCs, allowing for a fluid and responsive gaming experience. This approach not only enriches the gameplay but also provides an immersive and unpredictable environment that keeps players engaged and invested in the gaming world.

2) *Problem Context*: Introducing new features, such as super weapons in Action/FPS shooters or Nitro in racing games, to game opponents can be a technical challenge. One approach might be to add these features as new objects dynamically. However, this could lead to an overwhelming number of classes in the game system, causing complexity and potential development nightmares for developers. Game systems are extensive and intricate, and excessive classes and dependencies can severely impact the reliability and reusability of future titles. To address this complexity, the Decorator pattern offers an elegant solution. Instead of creating numerous classes for each combination of features, the Decorator pattern allows developers to add features dynamically at runtime without modifying existing classes. By employing decorators, the game system can enhance opponents with new functionalities as needed, such as attaching a super weapon or enabling a Nitro boost, all without creating a multitude of new classes. Implementing the Decorator pattern streamlines the development process and reduces the burden on developers. It promotes a modular approach, where features can be independently added or removed, making it easier to manage and maintain the game system. Additionally, the pattern enhances the reusability of existing classes, as decorators can be combined in various ways to create diverse opponents with unique attributes. By embracing the Decorator pattern, game developers can strike a balance between feature-rich gameplay and a manageable codebase[32]. This ensures a smoother development process, reduces complexities, and enables future titles to benefit from the flexible and extensible architecture. Ultimately, the Decorator pattern proves to be an indispensable tool in crafting compelling games with dynamic and customizable opponent features while preserving the scalability and maintainability of the game system.

3) *Solution*: In the context of Dynamic Difficulty Adjustment (DDA), the Decorator pattern proves invaluable for enhancing Non-Playable Characters (NPCs) with new weapons and upgrades, dynamically adapting to the gameplay difficulty. The SpecialWeapons class plays a vital role in providing specific characteristics, super weapons, or other elements that intensify the gaming experience. By incorporating more sophisticated weapons like RPG7s, the Decorator pattern empowers NPCs to challenge the player strategically. The function SpecifyRank() plays a crucial role in determining the rank of each NPC, influencing the type of weapons and upgrades they receive. Depending on the NPC's rank, the function returns a string representing the rank name, which serves as a key indicator for determining the appropriate weaponry to be added dynamically[33]. Fig. 3 shows that the Decorator pattern in this scenario allows the game system to scale NPC difficulty smoothly without the need for excessive class creation. Rather

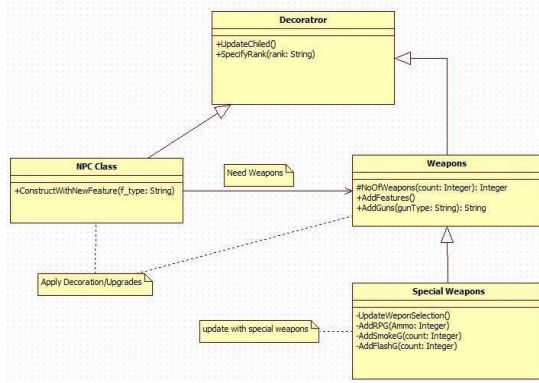


Fig. 3. Decorator pattern Class diagram

than hard-coding specific NPC types, the Decorator pattern enables the addition of new weapon functionalities on the fly, enhancing the versatility and reusability of existing classes. By combining various decorators, NPCs can be tailored to have unique attributes, further enriching the gaming experience. In summary, the Decorator pattern facilitates the agile and flexible augmentation of NPCs with upgraded weapons, ensuring that the gameplay remains challenging and engaging for players. This design strategy empowers game developers to easily fine-tune NPC attributes, creating a dynamic and ever-evolving gaming environment that captivates players with its depth and excitement.

4) *Consequences*: reducing class dependencies. This pattern proves invaluable in empowering NPCs with more potent weapons and upgrades without resorting to sub-classing. Instead, the Decorator pattern acts as a vital class that facilitates these enhancements by leveraging existing in-game upgrade features available in other classes, allowing for dynamic reuse and attachment to different objects. This approach streamlines the upgrade process, as it eliminates the need to create multiple subclasses for varying NPC attributes. Instead, the Decorator pattern provides a flexible and reusable solution, enhancing NPCs with additional functionalities on the fly. By reattaching available features to different objects dynamically, the system maintains its agility and modularity, facilitating smoother future updates and expansions. With the Decorator pattern, game developers can efficiently manage NPC upgrades and weapon enhancements without sacrificing system reliability or introducing unnecessary complexities. The result is a streamlined and efficient development process that bolsters the overall gameplay experience, providing players with an engaging and challenging gaming environment.

D. Consolidating Design Elements of the Framework

Dynamic Difficulty Adjustment (DDA) has been proven to enhance the gameplay experience in various scenarios, but its implementation can pose challenges in terms of development difficulty, complexity, and cost. This has led some experts to suggest that applying DDA in large-scale games may be a more practical approach. However, given the diversity of gaming projects and their unique requirements, a

generic framework based on object-oriented design patterns emerges as a promising alternative to existing approaches. Such a framework can offer greater flexibility, reusability, reliability, and shorter testing and bug tracking times. The research acknowledges that there is no one-size-fits-all solution for implementing DDA in different game projects. Considering this, the proposed generic framework aims to provide developers with more freedom and adaptability to tailor the DDA implementation to suit their specific scenarios. By accommodating diverse requirements and scenarios, the framework offers a versatile solution for enhancing gameplay experiences across a range of games. Design Patterns serve as proven standards in Object-Oriented Programming (OOP) projects. By basing the framework on well-established design patterns, the development process becomes more efficient and reliable. These patterns also reduce coupling and dependency between classes, leading to more maintainable and reusable software components. Additionally, the framework's design allows for quicker testing and bug tracking, resulting in a more streamlined development cycle. The proposed framework narrows its focus on three key design patterns that play crucial roles in DDA implementation: Observer, Strategy, and Decorator patterns. These patterns excel at decoupling classes and reducing the need for sub-classing. By incorporating these patterns, the framework effectively supports the application of DDA in object-oriented games, providing developers with the tools they need to create engaging and dynamic gameplay experiences. Fig. 4 illustrates the generic object-oriented framework that fully supports DDA. This framework is designed to be scenario-agnostic, providing developers with a blank canvas to tailor the DDA implementation according to their game's unique characteristics. The versatility of the framework allows more developers to utilize it, empowering them to modify and adapt the design to fit their specific user scenarios. Following is the description of design pattern-based classes in the proposed framework:

1) *Observer Class*: The Observer class serves as a crucial component in the framework, responsible for monitoring the player's states and notifying its child classes about any changes or updates. This dynamic communication enables the Observer class to trigger appropriate actions in response to the player's interactions or progress within the game. By effectively relaying information to its subscribers, the Observer class facilitates seamless coordination between various game elements, ultimately enhancing the gaming experience through real-time adjustments.

2) *Decorator Class*: In the proposed framework, the Decorator class plays a pivotal role in dynamically upgrading or downgrading the power and features of Non-Playable Characters (NPCs). Leveraging the feedback and updates provided by the Observer pattern, the Decorator class efficiently modifies the attributes and capabilities of NPCs during gameplay. Notably, this class offers the advantage of utilizing existing features from external sources without the need for creating specialized classes. As a result, the Decorator class empowers developers to implement versatile and adaptive gameplay

scenarios that cater to players of varying skill levels.

3) *Strategy Class*: The Strategy class is a key design element that facilitates the runtime interchangeability of algorithms, particularly for pathfinding purposes. With the dynamic nature of gameplay, different pathfinding algorithms may be required based on the game's progress and complexity. The Strategy class effectively encapsulates and manages these interchangeable algorithms, allowing the framework to seamlessly adapt and substitute algorithms based on the feedback received from the Observer class. By maintaining a repository of algorithmic strategies, the Strategy class enhances the framework's flexibility and scalability, enabling efficient gameplay adjustments without compromising performance. The proposed framework employs three design pattern-based classes to support Dynamic Difficulty Adjustment in video games. Through the Observer class, real-time monitoring and communication are achieved, empowering the system to respond dynamically to player actions. The Decorator class enables versatile upgrades and downgrades for NPCs, enhancing gameplay difficulty without the need for additional specialized classes. Meanwhile, the Strategy class facilitates algorithmic interchangeability, ensuring the framework can efficiently adapt to changing gameplay requirements. By combining these design pattern-based classes, the proposed framework offers a comprehensive and effective solution for enhancing gameplay experiences while maintaining a high level of flexibility and reusability.

V. DISCUSSION, LIMITATIONS, AND FUTURE RECOMMENDATIONS

In order to simplify and facilitate the implementation of Dynamic Difficulty Adjustment (DDA) in video games, a technical object-oriented framework has been developed, utilizing selected design patterns from the Gang of Four. The framework offers two distinct representations, each providing developers with flexible options for implementation.

- 1) The first representation involves individual design patterns, each represented by a class diagram and mapped to a game example. This approach highlights the dynamic nature of the proposed framework, allowing developers to use specific components as needed without the requirement to implement the entire framework. This flexibility ensures that developers can tailor the framework to suit their specific design requirements. For instance, if the Observer Pattern is required in the game's software design, developers can easily attach the relevant components from the presented framework without the need for redundant implementation.
- 2) The second representation offers a comprehensive usage of all three proposed patterns, forming a generic design for the framework. Unlike the individual mapped scenarios, this representation follows standardized naming conventions, making it simple and adaptable for new game implementations or scenarios where all three patterns are relevant.

By demonstrating how the framework handles the implementation of DDA, its flow has been clearly explained. The framework, when utilized as a whole, provides an effective solution for applying DDA in video games. The proposed framework presents an accessible and practical tool for game developers. Its design emphasizes flexibility and adaptability, catering to diverse project requirements. By integrating proven design patterns, this framework streamlines the implementation of DDA, enhancing the overall gaming experience for players and contributing to the advancement of game development practices. Despite the extensive significance of the proposed framework, certain limitations and potential areas for improvement should be acknowledged. It is important to note that while design patterns offer numerous advantages, some developers may still perceive them as increasing system complexity. As the proposed framework is based on design patterns, certain developers may find it not fully compatible with their projects. To address this concern, the design should be further refined to allow more customization and flexibility, accommodating different project requirements. Another limitation arises from the fact that some developers continue to develop games using non-object-oriented approaches. For these developers, the proposed framework may not be applicable. Future research should explore ways to bridge this gap and offer solutions that cater to a wider range of development methodologies. While the framework provides valuable support for Dynamic Difficulty Adjustment (DDA), developers should consider utilizing additional patterns or strategies, if applicable, to further enhance their projects. Integrating multiple patterns in combination can lead to even more robust and tailored solutions. Implementing DDA effectively using the right tools is expected to garner positive reactions from game reviewers and players. While it may take time for developers to fully adopt DDA, its widespread use is foreseeable as more developers recognize its benefits. The current design is focused on object-oriented systems and applies DDA to specific attributes of game components. However, potential improvements could involve supporting dynamic environmental customization, enabling intelligent behaviours of Non-Playable Characters (NPCs) in response to changing environmental conditions.

CONCLUSION

The proposed framework design, based on Object-Oriented Design Patterns (OODPs), offers a compelling solution to address the challenges of DDA implementation. However, it is essential to acknowledge that OODPs have their own limitations and may be better suited for large-scale projects. The justification for this framework design has been thoroughly examined, and its potential benefits and applications have been discussed. Moreover, the research has also highlighted the limitations and areas for future improvement of the proposed framework. While the design is tailored to accommodate the challenges of DDA, it is necessary to continue refining and optimizing the framework to make it more versatile and adaptable to different project requirements. In conclusion, the suggested

object-oriented framework for DDA can significantly enhance the gaming experience, catering to players of diverse skills and preferences, and ultimately leading to increased player satisfaction. As game development continues to evolve, the framework presented in this research stands as a valuable tool to promote the seamless integration of DDA, further contributing to the evolution and improvement of the gaming landscape.

REFERENCES

- [1] F. Vaudour and A. Heinze, "Software as a service: Lessons from the video game industry," *Glob. Bus. Organ. Excel.*, vol. 39, no. 2, pp. 31–40, 2020.
- [2] K. Xiaohan, M. N. A. Khalid, and H. Iida, "Player satisfaction model and its implication to cultural change," *IEEE Access*, vol. 8, pp. 184375–184382, 2020.
- [3] M. Zohaib, "Dynamic difficulty adjustment (DDA) in computer games: A review," *Adv. Hum.-Comput. Interact.*, vol. 2018, pp. 1–12, 2018.
- [4] J. Nielsen, *Usability Engineering*. San Diego, CA: Academic Press, 1993.
- [5] S. Xue, M. Wu, J. Kolen, N. Aghdaie, and K. A. Zaman, "Dynamic difficulty adjustment for maximized engagement in digital games," in *Proceedings of the 26th International Conference on World Wide Web Companion - WWW '17 Companion*, 2017.
- [6] M. P. Silva, V. do Nascimento Silva, and L. Chaimowicz, "Dynamic difficulty adjustment through an adaptive AI," in *2015 14th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*, 2015.
- [7] A. Rollings and D. Morris, *Game architecture and design: A new edition*. Upper Saddle River, NJ: New Riders Publishing, 2003.
- [8] Tan, C. I., Wong, Y. S., Lim, T. S. J., & Chong, W. S. (2014, October). The Link between Experiential Learning Using Games and the Dichotomy of the Psychological and Sociological Studies in Game Design. In *European Conference on Games Based Learning* (Vol. 2, p. 832). Academic Conferences International Limited.
- [9] R. Hunicke, "The case for dynamic difficulty adjustment in games," in *Proceedings of the 2005 ACM SIGCHI International Conference on Advances in computer entertainment technology*, 2005.
- [10] Y. Hao, S. He, J. Wang, X. Liu, Jiajian Yang, and W. Huang, "Dynamic Difficulty Adjustment of Game AI by MCTS for the game Pac-Man," in *2010 Sixth International Conference on Natural Computation*, 2010.
- [11] Eglesz, D. Fekete, I. Kiss, and O. E. Izso', "Computer games are fun? On professional games and players' motivations," *Educational Media International*, vol. 42, no. 2, pp. 117–124, 2005.
- [12] B. Bowman, N. Elmqvist, and T. J. Jankun-Kelly, "Toward visualization for games: Theory, design space, and patterns," *IEEE Trans. Vis. Comput. Graph.*, vol. 18, no. 11, pp. 1956–1968, 2012.
- [13] B. Cowley and D. Charles, "Behavlets: a method for practical player modelling using psychology-based player traits and domain specific features," *User Model. User-adapt Interact.*, vol. 26, no. 2–3, pp. 257–306, 2016.
- [14] D. Charles et al., "Player-centred game design: Player modelling and adaptive digital games," pp. 285–298, 2005.
- [15] L. Sha et al., "Creating appropriate challenge level game opponent by the use of dynamic difficulty adjustment," in *2010 Sixth International Conference on Natural Computation*, 2010.
- [16] G. K. Sepulveda, F. Besoain, and N. A. Barriga, "Exploring dynamic difficulty adjustment in videogames," in *2019 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON)*, 2019.
- [17] X. Cheng, J. Xiao, Q. Miao, and Y. Wang, "Design and implementation of a software architecture for 3D-DDA," *Sci. China Technol. Sci.*, vol. 58, no. 9, pp. 1604–1608, 2015.
- [18] C. V. N. Segundo, K. Emerson, A. Calixto, and R. P. Gusmao, "Dynamic difficulty adjustment through parameter manipulation for Space Shooter game," in *Proceedings of SB Games*, 2016.
- [19] M. Gonzalez-Duque, R. B. Palm, D. Ha, and S. Risi, "Finding game levels with the right difficulty in a few trials through intelligent trial-and-error," in *2020 IEEE Conference on Games (CoG)*, 2020.
- [20] S. He, J. Du, H. Chen, J. Meng, and Q. Zhu, "Strategy-based player modeling during interactive entertainment sessions by using Bayesian classification," in *2008 Fourth International Conference on Natural Computation*, 2008.
- [21] A. Darzi, S. M. McCrea, and D. Novak, "User experience with dynamic difficulty adjustment methods for an affective exergame: Comparative laboratory-based study," *JMIR Serious Games*, vol. 9, no. 2, p. e25771, 2021.
- [22] E. Post and C. Lincoln, *Advantages of using the object-oriented paradigm for designing and developing software*. 2001.
- [23] E. Gamma, R. Helm, R. Johnson, J. Vlissides, and C. Larman, *Design patterns: Elements of reusable object-oriented software with applying UML and patterns: An introduction to object-oriented analysis and design and the unified process*. London, England: Addison Wesley, 2003.
- [24] A. Kurmangali, M. E. Rana, and W. N. W. Rahman, "Impact of Abstract Factory and Decorator Design Patterns on Software Maintainability: Empirical Evaluation using CK Metrics," in *2022 International Conference on Decision Aid Sciences and Applications (DASA)*, IEEE, 2022, pp. 517–522.
- [25] H. K. Jun and M. E. Rana, "Evaluating the impact of design patterns on software maintainability: An empirical evaluation," in *2021 Third International Sustainability and Resilience Conference: Climate Change*, 2021.
- [26] E. Khonica, I. Medi, and M. E. Rana, "Empirically Tested Design Pattern Selection Framework Based on Developer Experience," *CompuSoft*, vol. 9, no. 9, pp. 3841–3847, 2020.
- [27] K. Hullett and J. Whitehead, "Design patterns in FPS levels," in *Proceedings of the Fifth International Conference on the Foundations of Digital Games*, 2010.
- [28] S. Bjork and J. Holopainen, *Patterns in game design (game development series)*. Charles River Media, Inc, 2004.
- [29] M. I. Chowdhury and M. Katchabaw, "Software design patterns for enabling auto dynamic difficulty in video games," in *2012 17th International Conference on Computer Games (CGAMES)*, 2012.
- [30] Starks, K. (2014). Cognitive behavioral game design: a unified model for designing serious games. *Frontiers in Psychology*, 5. <https://doi.org/10.3389/fpsyg.2014.00028>.
- [31] Nakamura, J., & Csikszentmihalyi, M. (2014). The concept of flow. In *Flow and the foundations of positive psychology* (pp. 239-263). Netherlands: Springer.
- [32] Kok S.H., Abdullah A., Jhanjhi N.Z., "Early detection of cryptoransomware using pre-encryption detection algorithm", *Journal of King Saud University - Computer and Information Sciences*, 34 (5), pp. 1984 - 1999, 2022. DOI: 10.1016/j.jksuci.2020.06.012
- [33] Gouda W., Almurafeh M., Humayun M., Jhanjhi N.Z., "Detection of COVID-19 Based on Chest X-rays Using Deep Learning", *Health-care (Switzerland)*, 10 (2), art. no. 343, 2022. DOI: 10.3390/healthcare10020343